

dr. Matej Šprogar

# Moj prvi program

*za spremembo ne bo Zdravo, svet!*

# Bistvo programiranja so podatki

Računalnik dela s podatki. To so lahko števila (vgrajeno v procesor), lahko pa so tudi logične vrednosti, znaki, stringi (vgrajeno v prevajalnik), dokumenti, piksli, slike, filmi (doda programer)... Kar so v osnovi vse števila!

Poskusimo **v računalnik shraniti** število 42:

Vprašanje je, KAM ga bomo shranili?

Opcije:

1. kalkulator
2. preglednica
3. urejevalnik besedil
4. ?

Za odgovor je pomembno, *kaj* želimo s tem številom v nadaljevanju... Verjetno bo večino stvari zmogla preglednica ali urejevalnik?

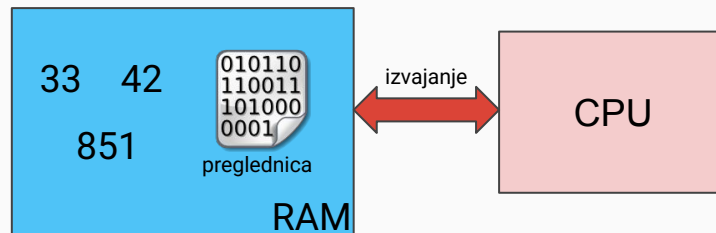
# Številka v preglednici

Ko v celico zapišemo 42, bo preglednica to takoj razumela kot število (to je konec koncev namen preglednic) in bo 42 zapisala v pomnilnik in kasneje še v datoteko na disk.

Z 42 lahko sedaj delamo zgolj to, kar dopušča program za preglednico.

Za uporabo preglednice ni potrebno niti vedeti, da je število shranjeno v pomnilniku, zato pa jo lahko uporabljajo tudi ne-programerji!

|  |     |  |  |    |    |  |  |
|--|-----|--|--|----|----|--|--|
|  |     |  |  |    |    |  |  |
|  | 851 |  |  |    |    |  |  |
|  |     |  |  |    |    |  |  |
|  |     |  |  | 42 | 33 |  |  |
|  |     |  |  |    |    |  |  |
|  |     |  |  |    |    |  |  |



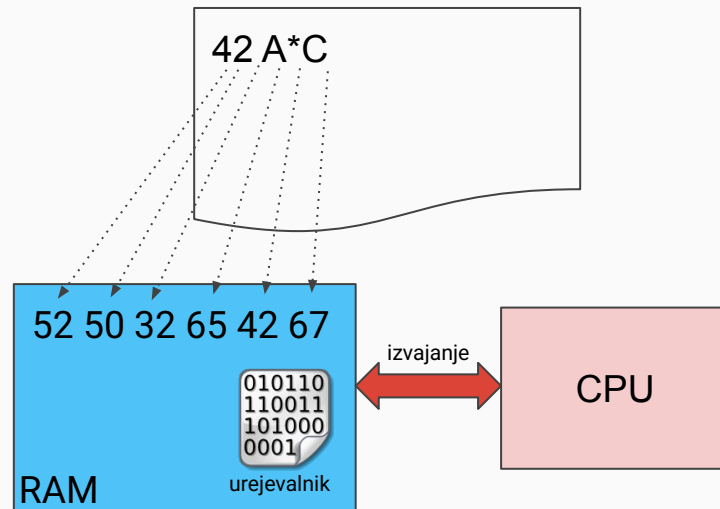
# Urejevalnik dela s črkami

Ko 42 napišemo v urejevalniku besedil, se naš vnos smatra kot **beseda** 42, ne pa kot **število** 42.

**Tip podatka je drugačen!**

Ker procesor dela samo s števili, se črke naše abecede preslikajo v števila (črka A je 65, B je 66...). Enako številke 0-9, le da ima, nasprotno pričakovanju, številka 0 kodo 48, 1 je 49, 2 je 50 itd. Presledek je recimo 32. (glej *Unicode*)

Tudi tokrat ne delamo *neposredno* z RAM-om!



# 42 v pomnilnik s programom

Če našim željam z 42 ne zadostijo ne preglednica, ne urejevalnik, ne kalkulator, ne brskalnik..., nam ostane samo še programiranje.

Procesorju lahko ukažemo vse, kar je sposoben narediti, s pomočjo programskih jezikov.

Želimo enako kot pri preglednici ali urejevalniku - jezik naj bo čim enostavnejši za uporabo!

Če želimo v pomnilnik zgolj vpisati število 42 kot navaden podatek, je potrebno sprejeti kar nekaj odločitev:

- 1) **kam točno** se naj število zapiše (da ne *prepišemo* česa drugega),
- 2) kako bomo do podatka **enostavno** dostopali (sicer se *izgubi* v poplavi drugih števil),
- 3) kako bo podatek **razumljen**, ko ga spet preberemo (kaj številka v pomnilniku *pomeni*/kako jo vidimo),
- 4) kako dolgo naj **ostane** število v pomnilniku (da ne *zaseda* prostora po nepotrebnem)
- 5) kako omejiti **dostop** do podatka (da nam ga kdo ne spremeni, pokvari)
- 6) ...

# Delo s podatki v programskem jeziku

Programski jezik mora omogočati vsaj tri stvari:

1. *branje podatka (iz pomnilnika v register),*
2. *izraze za računanje (v okviru registrov),*
3. *pisanje rezultata (iz registra v pomnilnik).*

Vse naj bo čim bolj enostavno in učinkovito.

Registrov je malo in so zato lahko poimenovani pomnilniški elementi znotraj procesorja. Za delo z **ogromnim** delovnim pomnilnikom pa moramo poznati *lokacijo* in *obliko* podatka (*neugodno!*). Ugodno bi bilo, če bi lahko poimenovali tudi koščke delovnega pomnilnika...

Namesto nizkonivojskega in dlakocepskega celo število v 4 bytih na lokaciji `0x00FBF793` bi raje napisali zgolj (recimo) `visina`.

Jeziki *C++*, *Java* in *C#* ustvarijo povezavo med poljubnim imenom, tipom in lokacijo avtomatsko dodeljenega koščka prostega pomnilnika s tim. **deklaracijskim** ukazom:

```
tip ime_podatka;
```

kjer je *tip* ime vgrajenega (`int`, `double`..) ali programerjevega (`Pivo`..) podatkovnega tipa.

```
int visina;  
Pivo moja_pijaca;
```

# Spremenljivka

Dostop do podatka v pomnilniku je mogoč le, če poznamo njegov naslov in velikost. Prevajalnik lahko oboje nadomesti z enostavnejšim imenom, ki ga določimo skupaj s tipom podatka v deklaraciji spremenljivke.

Tip podatka določa:

- (1) potrebno količino pomnilnika,
- (2) kodiranje podatka.

Podatek lahko tako beremo in pišemo "preko" njegovega imena.

Lahko ga kadarkoli spremenimo, od tu tudi ime *spremenljivka*.

Python ne zahteva informacije o tipu:

```
odgovor = 42
```

C++, Java in C# so nasprotno strogi:

```
int odgovor;  
odgovor = 42;
```

Od deklaracije naprej (navzdol) lahko kadarkoli uporabimo spremenljivko z imenom `odgovor`, ki sedaj pomeni število zapisano nekje(!) v pomnilniku.

V deklaraciji lahko najavimo več spremenljivk hkrati, lahko jih tudi *inicializiramo*, vse v istem ukazu:

```
int odgovor = 42, vprašanje = 7;
```

# Branje in pisanje spremenljivk

V spremenljivko **pišemo**, če jo uporabimo na levi strani prireditvenega stavka - vpisana bo *vrednost* desne strani prireditvenega stavka.

Spremenljivko **beremo**, ko uporabimo njeno ime na desni strani prireditve. Takrat procesor prebere in uporabi njeno vrednost iz pomnilnika.

**Prireditveni stavek je vsak ukaz, ki vsebuje enačaj '=':**

*left\_hand\_side = right\_hand\_side;*

```
// najprej deklaracija spremenljivk brez pisanja  
int alfa, beta;
```

```
// pisanje v spremenljivko na levi strani  
alfa = 3; // alfa ← 3  
beta = alfa; // beta ← alfa
```



```
// nesmiselno je, da rezultata ne shranimo  
3+5; // 3+5 → 8 NAPAKA?  
alfa+7; // alfa → 3, 3+7 → 10 NAPAKA?
```

```
// koristno računanje izrazov na desni strani  
alfa = beta; // beta → 3, 3 → alfa  
beta = alfa+1; // alfa → 3, 3+1 → beta
```

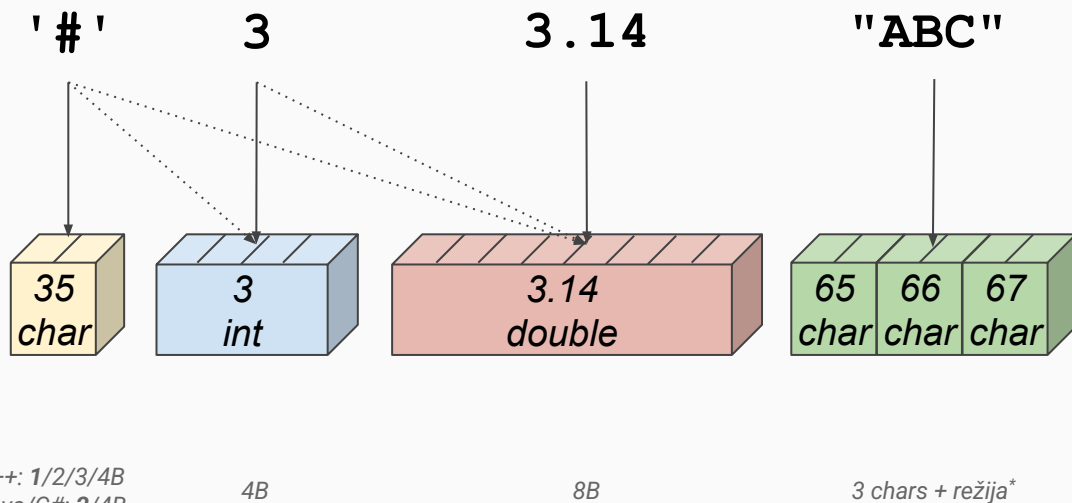
```
// leva stran prireditve mora biti zapisljiva  
alfa+1 = 3; // NAPAKA: lhs ni spremenljivka
```

# Vsebina spremenljivke

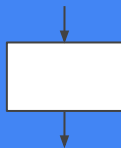
Spremenljivka je košček fizičnega pomnilnika, ki lahko hrani natančno **eno** vrednost določenega **tipa**.

Če je recimo spremenljivka namenjena celim številom (*int*), vanjo ne moremo shraniti realnega števila (*double*), ki zahteva dvakrat več prostora in tudi drugačno organizacijo bitov...

Lahko pa obratno celo število 3 shranimo v škatlo za realna števila (3.0), a izgubimo informacijo o *celoštevilski* naravi podatka 3.



# Spremenljivke in gnezdenje



V blok lahko vgnezdimo tudi druge bloke, ki imajo dostop do elementov zunanega bloka; obratno ne velja!

Gnezdenje najpogosteje uporabljamo v okviru odločitvenega stavka, kjer z omejeno vidljivostjo notranjih spremenljivk dosežemo varnejšo kodo, saj zunanja koda ne more *pomotoma* brati vsebine, ki ji ni namenjena.

Ekstrem: če je spremenljivka na najvišjem zunanjem nivoju, nastane hud *problem globalne* spremenljivke.

**Vgnezdeni** blok lahko uporablja vse elemente *zunanjega* bloka, njegovi lastni elementi pa navzven niso vidni zaradi varnostnih\* razlogov.

```
{  
    int zunaj = 0;  
    {  
        int notri = 1;  
        notri = notri + zunaj;  
    }  
    zunaj = notri + zunaj; // NAPAKA  
}
```

# Literalna konstanta

Najlažje je delati s podatki, ki se med izvajanjem programa **ne** spremenijo.

Tipično so to v programu direktno navedene vrednosti (tim. *literali* - števila, logične vrednosti, črkovne konstante, nizi znakov...) ali rezultati nespremenljivih izrazov.

Literali so konstantne vrednosti, ki jih nikoli več ne bomo rabili, zato jih nima smisla ne poimenovati ne shraniti.

```
42
40+2
3.14159
true
'A'
"abc"
...
```

Tako kot spremenljivka, ima tudi konstanta svoj podatkovni tip, ki ga prevajalnik določi na podlagi oblike zapisa konstante (število? decimalke? narekovaji? pripona?...)

Nekaj literalov v kodi lahko olajša berljivost, če pa se ponavljajo ali je več podobnih, lahko otežijo vzdrževanje izvirne kode v prihodnje (ob morebitnih spremembah in popravkih). Takrat je smiselno uporabiti...

# Simbolna konstanta

Če bomo vrednost neke konstante v prihodnosti mogoče spremenili ali pa se v kodi večkrat ponovi, jo je smiselno **poimenovati**.

Vedenje, da se nekaj ne bo spremenilo, je namreč koristen pripomoček za preprečevanje napak zaradi nevednosti, površnosti, nepazljivosti ali utrujenosti programerja.

Ob deklaraciji spremenljivke dodatno uporabimo ustrezno rezervirano besedo, ki iz spremenljivke naredi "nespremenljivko" - konstanto.

Python ne pozna "prave" konstante:

```
ODGOVOR = 42
```

Java:

```
final int ODGOVOR = 42;
```

C++ in C#:

```
const int ODGOVOR = 42;
```

Sedaj lahko konstanto beremo (uporabimo):

```
int polovica = ODGOVOR / 2;
```

Pri morebitnem poskusu spreminjanja konstante bo prevajalnik sedaj javil *napako*:

```
ODGOVOR = 43;           // NAPAKA: prepovedano
```

# Slovnica in slovar jezika

Sedaj smo že videli, da prevajalnik pričakuje ukaze oblikovane skladno z nekimi pravili. Podobno, kot ima slovenščina zapovedano slovnico *in* besednjak, imajo tudi programski jeziki slovnična pravila in nabor *rezerviranih besed*.

Rezervirane besede lahko skladno s slovnico uporabimo samo na določenih mestih v programu in imajo vnaprej definiran pomen. "*jezika programskega slovnica enostavna zelo je*" in se jo lahko hitro\* naučimo.

Za razliko od slovenščine pa lahko mnogo programskih jezikov še **razširimo** z lastnimi besedami - ustvarimo svojo govorico, ki jo prevajalnik "razume"!

# Prevajanje

Prevajalnik ima določena pričakovanja, ki se med jeziki seveda razlikujejo. Java in stari C# recimo pričakujeta, da bo vsa koda objektna, zato vsilita za objektno programiranje potrebno minimalno strukturo z glavnim (*main*) blokom kode, ki naj se sproži ob zagonu programa. C++ nasprotno zahteva le *main*, novejši C# pa niti tega več ne... Vsi trije naredijo *ogromno* v ozadju.

Programerskim napakam v slovnici, ki jih zazna prevajalnik, pravimo slovnične (***sintaktične***) napake, težavnejše vsebinske (***semantične***) napake v algoritmu pa se pokažejo šele med delovanjem programa!

# Prvi program

Program, ki spremeni zgolj svoj pomnilnik, ni preverljiv, zato bomo v šolskih primerih pogosto prikazali rezultate na ekranu.

Zaenkrat delamo le z vsebino označene škatle; zunanjo kodo pustimo pri miru...

hello.cpp:

```
#include <iostream>
int main() {
    const int odgovor = 6*7;
    std::cout << odgovor;
}
```

hello.java:

```
class Program {
    public static void main(String[] args) {
        final int odgovor = 6*7;
        System.out.print(odgovor);
    }
}
```

hello.cs:

```
class Program {
    static void Main(string[] args) {
        const int odgovor = 6*7;
        System.Console.Write(odgovor);
    }
}
```

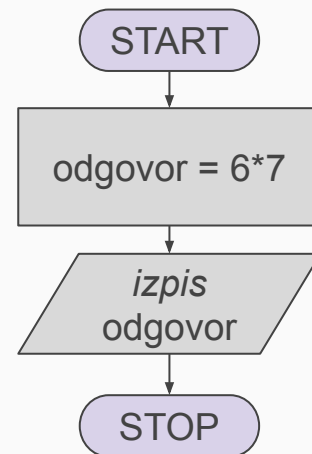
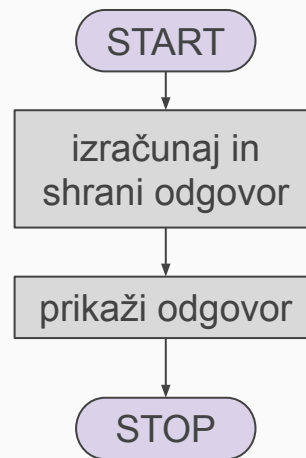
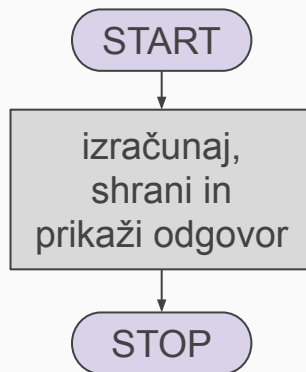
# Razlaga

Prvi ukaz v kodi pomeni definicijo konstante *odgovor*, ki ji že prevajalnik izračuna vrednost 42.

Drugi ukaz izpiše vrednost konstante na tim. *konzolo* (primitiven ekran, ki je sposoben prikazati zgolj besedilo). Dejansko uporabimo za to namenjen sistemski podprogram\* (o tem kasneje).

Nalogo izpisa 42 smo razgradili in opravili v dveh korakih:

1. Izračunali smo odgovor in ga shranili v pomnilnik
2. Shranjeno vrednost smo posredovali *podprogramu* za izpis.



# Osnovni gradniki kode

Trije glavni splošno-namenski jeziki današnje dobe (**C++** (1985), **Java** (1995) in **C#** (2000)) so močno povezani, dejansko Java in C# izhajata iz prvega, C++ pa iz jezika **C** (1972).

Skoraj identične **temeljne** lastnosti kode so:

1. komentarji (`/* */` in `//`)
2. zaključevanje stavkov s podpičjem;
3. strogo preverjanje tipov
4. dovoljena poimenovanja
5. način branja in pisanja spremenljivk
6. bloki kode v zaviti oklepajih `{ }`
7. osnovni *tipi*, izrazi, operatorji
8. sintaksa strukturiranih elementov (sekvenca, selekcija, zanke, podprogrami)
9. koncept glavne funkcije\*
10. ...

```
/* prevajalnik vsebino komentarjev ignorira;
   namenjeni so zgolj človeškemu bralcu programa */
int alfa;           // deklaracija spremenljivke
alfa = 0;           /* piši 0 v alfa; 0 → alfa */

/* nova spremenljivka */           double pi = 3.14;
/* beri alfa & prepisi pi */      pi = alfa;

// dva stavka združena v en blok kode
{
    alfa = 3;
    pi = 3.14159;
}

/* in sedaj še nekaj sintaktičnih napak */
alfa = pi;           // NAPAKA: 3.14 → 3.0?
beta = 3;            // NAPAKA: kaj je beta?
42 = alfa;           // NAPAKA: 42 ni spremenljivka
int 7ka = 7;         // NAPAKA: slabo ime
```

# Poimenovanje

Pri izbiri imena moramo upoštevati:

1. prvi znak naj bo črka ali vezaj
2. sledijo lahko črke, številke, vezaji
3. velikost črk je pomembna
4. prepovedani so operatorji, ločila
5. ime ne sme biti rezervirana beseda

Dobro poimenovanje je **kritično** za razumljivost algoritma; žal je pogosto tudi najbolj podcenjena faza kodiranja.

Notacije pomagajo, a niso rešitev  
(*camelCase*, *PascalCase*, *snake\_case*)

```
// primeri veljavnih imen
```

```
dobro_ime, dobroIme, x, spremenljivka3, _a,  
številoElementov, zeloDolgoImeKiPaJeŠeVednoVeljavno
```

```
// primeri neveljavnih imen
```

```
6tinivo, 2ka, Nova vrednost, pazi!, a#2, .a, -cena
```

```
// nekaj dobrih imen
```

```
x, številoElementov, radij, visina, povprečna_teza
```

```
// nekaj slabih imen
```

```
x, bool, r, v, memberIntVisinaVm
```

# Osnovni tipi

Osnovni (enostavni) podatkovni tipi so vgrajeni v prevajalnike; skrbijo za en podatek. Sestavljeni tipi so le sestavljanke iz več osnovnih tipov.

Osnovni tipi zajemajo:

1. cela števila
2. realna števila
3. logične vrednosti
4. znake (črke)

Vsak tip ima svojo **zalogo vrednosti** in svoj nabor **operacij**, ki jih lahko izvede.

**Zaloga vrednosti** so vse vrednosti, ki jih lahko spremenljivka nekega tipa zavzame.

Na primer tip `int` opisuje cela števila in zavzema tipično 4 byte spomina, zato lahko v spremenljivko tega tipa shranimo  $2^{32}$  različnih celih števil, od  $-2^{31}$  do  $2^{31}-1$ ; števila  $2^{31}$  recimo vanj že ni več mogoče shraniti.

Če število *ne spada* v zalogo vrednosti, ga ni mogoče shraniti *pravilno* - pride do tim. prekoračitve (*overflow*):

```
sbyte x=127; x=x+1; // x je -128
```

# Operacije nad tipi

Osnovni tipi so namenjeni *računanju*, zato obvladajo vse klasične aritmetične operacije in še mnogo drugega. Večino od njih (plus, minus...) zna na bitnem nivoju narediti procesor, ostale pa lahko prevajalnik pripravi v programski obliki (sin, log...).

Glavna ideja je, da so nad določenimi tipi podatkov dovoljene zgolj nekatere operacije - da prevajalnik ne dovoli nečesa, kar je v osnovi *zgrešeno* ali bi lahko dalo *nepričakovan* rezultat.

Vsak podatkovni tip nudi svoj nabor operacij, ki jih lahko uporabimo nad podatki tega tipa. Recimo seštevanje obstaja za cela in realna števila ( $40+2 \rightarrow 42$ ), ne pa tudi za logične vrednosti ( $true+false \rightarrow ?$ ).

Aritmetične operacije:  $+$   $*$   $-$   $/$   $\%$

Logične operacije:  $<$   $>$   $==$   $>=$   $<=$   $\&\&$   $||$   $^$   $!$

Bitne operacije:  $\&$   $|$   $^$   $\sim$   $>>$   $<<$

Operatorji imajo **prioriteto** (*operator precedence*) in **asociativnost** (večinoma od leve proti desni:

$a+b+c \equiv (a+b)+c$ ; obratno velja za priredbo

$a=b=c \equiv a=(b=c)$ , predznake, ...

# 1 Cela števila

Obseg zaloge vrednosti je odvisen od velikosti podatkovnega tipa.

V izvorno kodo programa jih lahko najenostavneje vstavimo kot literalne konstante v desetiški ( $42 = 4 \cdot 10 + 2 \cdot 1$ ), šestnajstiški ( $0x2a = 2 \cdot 16 + 10 \cdot 1$ ), osmiški ( $052 = 5 \cdot 8 + 2 \cdot 1$ ) ali binarni obliki ( $0b101010 = 1 \cdot 32 + 0 \cdot 16 + 1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1$ ) zapisa.

Opomba: V C++ pod Win64 sta *int* in *long* dolga 32, *long long* pa 64 bitov.

Praktično vsi tipizirani jeziki ponujajo osnovni tip *int* (*integer*), ki je na modernem 64-bitnem procesorju tipično dolg 32 bitov, daljši *long* (*long int*) 64 in krajši *short* (*short int*) 16 bitov. Za 8 bitno predznačeno število ima Java tip *byte*, C# tip *sbyte*, C++ pa *int8\_t*. Java nima\* podpore za nepredznačena števila, C++ in C# pa s predpono: npr. *unsigned int*.

Zaloga vrednosti za nepredznačenih 8 bitov je tako  $0 \div 255$ , za predznačene pa  $-128 \div 127$ .

# 1.1 Operacije s celimi števili

Nad celimi števili so definirani vsi klasični matematični operatorji, ki večinoma delajo z dvema številoma.

Rezultat aritmetičnih operacij je vsaj `int` ali `long` (Java/C#) oz. *najširši* v računu uporabljen tip (C++).

Bitne operacije omogočajo delo s posameznimi biti v številu. Operacije vrnejo število!

Logične operacije preverjajo in primerjajo števila in rezultirajo v logičnem rezultatu (v nadaljevanju).

```
1 + 2 * 3      // 7
6 * (3 + 4)    // 42
20 / 3         // 6
20 % 3         // 2
```

**Bitne operacije** (`lshift <<`, `rshift >>`, `and &`, `or |`, `xor ^`, `not ~`):

```
42          // 42  0b0...0000101010
42<<3       // 336 0b0...0101010000
42&0b111    // 2   0b0...0000000010
42|0b111     // 47  0b0...0000101111
42^0b111     // 45  0b0...0000101101
~42          // -43 0b1...1111010101
```

## 1.2 Dodatni operatorji nad celimi števili

Nad celimi števili izvajamo različne aritmetične, logične in bitne operacije. Izrazi pogosto postanejo dolgi in neberljivi, zato so se uveljavili dodatni inovativni operatorji. Vseeno je, ali uporabimo navaden ali krajši zapis.

Operatorja ++ in -- oba spremenita original, hkrati pa v drugi račun **posredujejo** novo (prefiks) ali staro (postfiks) vrednost spremenljivke. Praviloma je pomembno, katerega izberemo.

Za prištevanje številu  $x$  obstaja bližnjica v obliki operatorja `+=` in sicer lahko namesto `x = x + nekaj`; **pišemo kar** `x += nekaj`; Obstajajo tudi `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`.

Še pogostejše je prištevanje 1 v kombinaciji z drugimi računi:

```
x += 1;  
novi_x = x;
```

ali krajši prefiks:

```
novi_x = ++x;
```

```
stari_x = x;  
x += 1;
```

ali krajši postfiks:

```
stari_x = x++;
```

## 2 Realna števila

Predstavitev realnega števila po IEEE 754 standardu lahko shrani števila v določenem **obsegu** in z določeno **natančnostjo**.

Omejena količina bitov lahko hrani le omejeno količino števil, realnih števil pa je *neskončno* mnogo. To pomeni, da računalnik ne bo nujno hranil prave vrednosti, **lahko** bo tudi **približek**!

Recimo števila 0.1 ni mogoče shraniti natančno, *float* število 16777217f se shrani kot 16777216f itd.

<https://www.h-schmidt.net/FloatConverter/IEEE754.html>

Manjšo natančnost nudi tip `float`, ki lahko v 32 bitih shrani največ  $2^{32}$  števil; razporedi jih v predznačene vrednosti med  $1.18 \cdot 10^{-38}$  in  $3.4 \cdot 10^{38}$  na 6-7 decimalnih mest natančno. Float konstante pišemo s pripono 'F' ali 'f':  
`3.14F`.

**Privzeta** natančnost realnih konstant je zato višja, dvojna (`double`), ki s 64 biti kodira 15 decimalnih mest od  $2.23 \cdot 10^{-308}$  do  $1.79 \cdot 10^{308}$ .

Pretvorba je avtomatska samo v eni smeri:

```
double pi = 3.14F; // float -> double
```

```
float pi = 3.14; // NAPAKA OK C++
```

# 3 Logične vrednosti

Logična vrednost lahko zavzame samo eno od dveh možnih vrednosti: **false** in **true**.

Logični operatorji delajo z logičnimi ali tudi drugimi vrednostmi in kot rezultat vrnejo logično vrednost.

Prevajalnik dovoli pretvorbo števila v logično vrednost po pravilu, da 0 ustreza *false*, ne-nič pa *true*!

C++ pretvori števila v logično vrednost po pravilu  $0 \equiv \text{false}$ , sicer *true*. Java in C# tega ne dopuščata!

V spremenljivki tipa `bool` (Java `boolean`) lahko hranimo le `false` ali `true`.

Logične operacije obsegajo primerjave števil (`<`, `<=`, `>`, `>=`, `==`, `!=`) z logičnim rezultatom in logične funkcije nad logičnimi vrednostmi: `&&` **and**, `||` **or**, `^` **xor**, `!` **not**.

| A           | B           | !A          | A && B      | A    B      | A ^ B       |
|-------------|-------------|-------------|-------------|-------------|-------------|
| false       | false       | <b>true</b> | false       | false       | false       |
| false       | <b>true</b> | <b>true</b> | false       | <b>true</b> | <b>true</b> |
| <b>true</b> | false       | false       | false       | <b>true</b> | <b>true</b> |
| <b>true</b> | <b>true</b> | false       | <b>true</b> | <b>true</b> | false       |

## 4 Znakovni tip

Medtem ko Java in C# uporabljata tip **char** za UTF-16 znake, uporablja C++ UTF-8 znake. Obe izbiri imata svoje prednosti in slabosti.

Vse znakovne literale pišemo v enojnih narekovajih. Če želimo zapisati katerikoli Unicode znak, ga lahko zapišemo s šestnajstiško kodo s štirimi števki in predpono `\u`.

Črka je v osnovi indeks znaka v Unicode tabeli in jo lahko uporabljamo kot številko, npr. `'0' == 48`, `'A' == 65`...

```
int mali_j = 'j';
```

Nizke številke so namenjene posebnim, krmilnim znakom: `'\n'`, `'\r'`, `'\t'`, `'\\'`, `'\b'` ...

Črka je način, kako številko, s katero dela procesor, preslikamo v človeku prijaznejšo obliko. Recimo število 106 lahko predstavlja 106. znak v Unicode tabeli, to je črko *j*, ki jo lahko zato zapišemo vsaj na dva enakovredna načina: `'j'` in `'\u006a'`.

```
char črkaMaliJ = '\u006a';
```

Lahko pa uporabimo izrecno *pretvorbo* (*cast*), ki prevajalniku ukaže, da želimo število 106 obravnavati kot *znak*:

```
char črkaMaliJ = (char)106;
```

## Kje so torej razlike med C++, Javo, C#?

C++ je namenjen programiranju visokozmogljivih in varnostno kritičnih sistemov (hard-real time, operacijski sistemi, knjižnice, gonilniki, igre...), Java in C# pa sta osredotočena na hitrejšo izdelavo predvsem poslovnih (strežniških in client) aplikacij z ne tako strogimi časovnimi ali prostorskimi zahtevami.

Izbira med njimi ni enostavna zaradi preobilice *megle* in *verskih vojn* na internetu.

Pogosto velja, da je najbolje uporabiti tisti jezik, v katerem lahko delamo najhitreje. Redki veliki projekti so enoznačno izdelani zgolj z enim jezikom.

Primeri C++ projektov: Adobe suite, Amadeus, Google, JVM, MS Office, MySQL, OS-i\*, SpaceX, Unity, Unreal...

Primeri Java/C# aplikacij: Hadoop, LinkedIn, Minecraft, Android Apps, Web Apps, Data Server Apps, UnityScript...

# Podprogrami

Ponavljajoče se večje bloke kode lahko smatramo spet kot programe v okviru programa, to je *podprograme*. V C++, Javi in C# so strukturirani podprogrami poimenovani različno, pa čeprav gre v osnovi za skoraj isto stvar. Če C++ še loči med *funkcijo* in *metodo*, je v Javi in C# vse le še zgolj metoda, saj slednja jezika ne omogočata neodvisnih funkcij. Besede *pod-program*, *rutina*, *procedura*, *funkcija* in *metoda* tako vse opisujejo **isti koncept**, razlike so v načinu uporabe.

Ampak podprogrami si zaslužijo svoje poglavje...

*Programming Is Not About Typing,  
It's About Thinking.*

Rich Hickey